Ex 4: Comparative Analysis of Large Language Models

Learning Objective:
Develop a Python program that performs a comparative analysis of Large Language Models by analyzing the text-generation performance of OpenAI, Gemini, and Hugging Face models.

Steps:

1. Import Required Libraries (OpenAI, HuggingFace Transformers, Gemini API)
2. Configure OpenAI key and Google Gemini API key. (This key should be personal. Do not share with anyone).
3. Initialize the Models (OpenAI, Gemini, and Hugging Face).
4. Write function to send prompt and receive response from OpenAI, Gemini, and Hugging Face models
5. Provide a common text prompt. (e.g., “Write a haiku about AI and nature”).
6. Generate Outputs by sending the same prompt to all three models and collect their responses.
7. Display the Model Responses individually. Use Markdown / HTML formatting to display results clearly.
8. Construct an evaluator. (For evaluator you can use Gemini) and produce ranks.
9. Display the ranked model names in best → worst order.

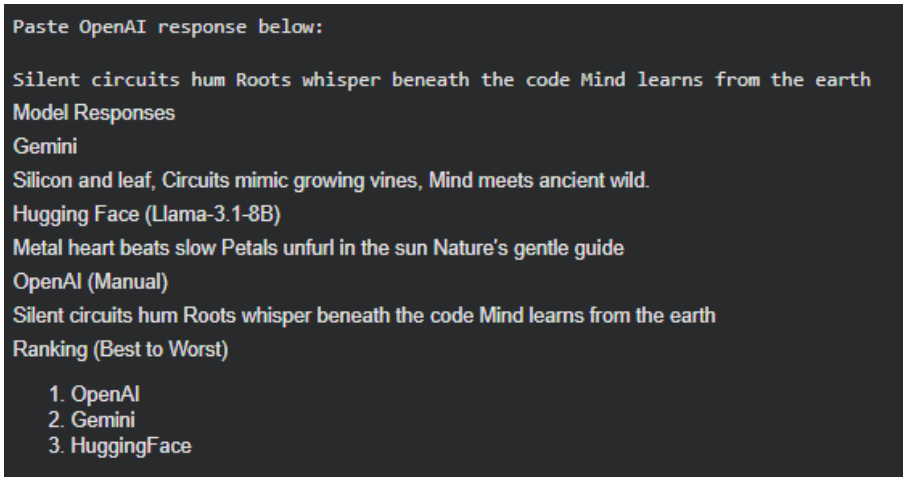
Program Code:

```
import google.generativeai as genai
import requests
import time
from IPython.display import display, Markdown

GEMINI_API_KEY = "AIzaSyAUo94tOAIAzv6VK5wa2v5eJsgdl86LIso"
HF_API_KEY = "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"

genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-3-flash-preview")

HF_MODEL = "meta-llama/Llama-3.1-8B-Instruct:cerebras"
HF_URL = "https://router.huggingface.co/v1/chat/completions"
hf_headers = {
    "Authorization": f"Bearer {HF_API_KEY}",
    "Content-Type": "application/json"
}

def query_gemini(prompt):
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Gemini Error: {e}"

def query_huggingface(prompt):
    payload = {
        "model": HF_MODEL,
```

```
        "messages": [{"role": "user", "content": prompt}],
        "max_tokens": 80,
        "temperature": 0.7
    }

    for _ in range(3):
        try:
            response = requests.post(HF_URL, headers=hf_headers, json=payload)

            if response.status_code == 200:
                data = response.json()
                return data["choices"][0]["message"]["content"].strip()

            elif response.status_code == 503:
                time.sleep(5)

            else:
                return f"HF Error {response.status_code}: {response.text}"

        except Exception as e:
            return f"HF Exception: {e}"

    return "HF model loading timeout."

prompt = "Write a haiku about AI and nature"

gemini_output = query_gemini(prompt)
hf_output = query_huggingface(prompt)

print("\nPaste OpenAI response below:\n")
openai_output = input()

display(Markdown("Model Responses"))

display(Markdown("Gemini"))
display(Markdown(gemini_output))

display(Markdown("Hugging Face (Llama-3.1-8B)"))
display(Markdown(hf_output))

display(Markdown("OpenAI (Manual)"))
display(Markdown(openai_output))

def evaluate(prompt, responses):
    eval_prompt = f"""
You are an evaluator.

Prompt:
{prompt}
```

```
Responses:
1. OpenAI: {responses['OpenAI']}
2. Gemini: {responses['Gemini']}
3. HuggingFace: {responses['HuggingFace']}

Evaluate based on:
- Creativity
- Relevance
- Fluency

Return ONLY ranking:
1. ModelName
2. ModelName
3. ModelName
"""
    try:
        result = model.generate_content(eval_prompt)
        return result.text.strip()
    except Exception as e:
        return f"Evaluation Error: {e}"

responses = {
    "OpenAI": openai_output,
    "Gemini": gemini_output,
    "HuggingFace": hf_output
}

ranking = evaluate(prompt, responses)

display(Markdown("Ranking (Best to Worst)"))
display(Markdown(ranking))
```

Output:



Learning Outcome:
Students will be able to integrate and interact with OpenAI, Gemini, and Hugging Face APIs and evaluate model outputs based on relevance.